Key Design Decisions

A. How environmental spawn effects are dispatched when creatures enter the map

Design 1: Each spawner (Hole, Vent, tree, infection status) directly applies the spawn effect inline after placing the creature. For example, Hole contains code to increase Undead max HP, drop worker items for Slime, etc.

Pros	Cons
Simple and direct; no additional classes needed.	Every spawner must duplicate the spawn-effect logic for each creature type, violating the DRY principle.
	Adding a new creature type requires modifying every spawner class, violating the Open-Closed Principle.
	Spawners become tightly coupled to creature-specific behaviour, violating the Single Responsibility Principle.
	Other REQs (trees, infection, Alien Cubes) that spawn creatures must also duplicate the same effect logic.

Design 2: Use a SpawnEffectManager singleton with a registry pattern. Each creature type registers a SpawnEffect (functional interface) at initialisation. After any spawner successfully places a creature, it calls SpawnEffectManager.onCreatureSpawned(), which looks up and applies the registered effect by creature class.

Pros	Cons
Spawn effects are defined once per creature type and applied consistently regardless of the source (Hole, Vent, tree, infection), satisfying the DRY principle.	Requires an additional manager class and interface, adding structural complexity.
New creature types can register their own effects without modifying any existing spawner or effect class (Open-Closed Principle).	The registry lookup uses Class as a key, which does not support subclass matching (exact class match only).
Spawners only know how to create and place creatures; they have no knowledge of what environmental reactions occur (Single Responsibility Principle).	The indirection through a singleton may make the control flow less immediately visible.

The SpawnEffect functional interface provides a single integration point for all systems across multiple REQs (REQ3 trees, REQ2 Alien Cubes, REQ4 spawners and infection) to call.	

B. How the Parasite discovers infectable targets polymorphically

Design 1: InfectBehaviour uses instanceof checks to test each adjacent actor and item against known infectable types (ContractedWorker, Undead, Lantern, Cookies) and applies the appropriate infection directly.

Pros	Cons
No additional interface needed; uses existing concrete types.	Tightly couples InfectBehaviour to every infectable class, violating the Dependency Inversion Principle.
	Adding a new infectable target (e.g., a new item from another REQ) requires modifying InfectBehaviour, violating the Open-Closed Principle.
	The use of instanceof is considered an anti-pattern in OOP as it bypasses polymorphic dispatch.

Design 2: Create an Infectable interface with an onInfected(Actor, GameMap) method. Each target implements it and defines its own infection response. InfectBehaviour discovers targets using the engine's getActorAs(Infectable.class) and getItemsAs(Infectable.class), which leverage GameEntity.asCapability() for polymorphic lookup.

Pros	Cons
InfectBehaviour depends only on the Infectable abstraction, not on any concrete class (Dependency Inversion Principle).	Each infectable class must implement the interface, adding a method to multiple existing classes.
New infectable targets can be added by implementing Infectable and registering nothing else — the Parasite will automatically discover them (Open-Closed Principle).	The Infectable interface works through asCapability(), which requires it to be an interface or abstract class.

Each target defines its own infection response in onInfected(), following the Single Responsibility Principle.	
Leverages the engine's existing asCapability() system (getActorAs, getItemsAs) for zero-cost discovery, reusing established infrastructure rather than inventing ad-hoc mechanisms.	

C. How different infection effects are applied to different host types

Design 1: Separate status classes per host category (WorkerInfectedStatus, LanternInfectedStatus, CookieInfectedStatus). Each host's onInfected() method creates and attaches the appropriate status. The engine's tickStatuses() mechanism ticks them automatically. Each status class contains the per-turn logic for its host type (e.g., WorkerInfectedStatus deals damage and spawns Parasites, LanternInfectedStatus drains oil).

Pros	Cons
Each status class handles exactly one infection effect (Single Responsibility Principle).	More classes to create and maintain (three instead of one).
No instanceof or type-checking needed; the host polymorphically selects the correct status in its onInfected() method.	The status classes must hold references to concrete host types (e.g., LanternInfectedStatus stores a Lantern field), creating tight coupling from the status layer to specific host classes.
	Status classes reach into host internals via public getter/setter methods (e.g., drainOil(), reduceCookies()), violating encapsulation by forcing hosts to expose internal state.
	Structural similarity between the status classes leads to duplication (e.g., the Parasite-spawning helper method is duplicated in WorkerInfectedStatus and CookieInfectedStatus).
	Adding a new infectable host requires both implementing Infectable and creating an additional status class, increasing the cost of extension.

Design 2: A single generic InfectedStatus class that delegates all per-turn logic to the host entity via the Infectable interface. The Infectable interface is expanded with two additional methods: onInfectionTick(Location) for per-turn behaviour, and isInfectionActive() for determining when the infection ends. InfectedStatus stores a reference to the Infectable host and calls host.onInfectionTick() in tickStatus() and host.isInfectionActive() in isStatusActive(). Each host implements these methods internally, managing its own state changes without exposing internal details.

Pros	Cons
Hosts fully encapsulate their own infection behaviour. Internal state (oil, cookies, HP) is managed privately without public getters/setters, following the Tell Don't Ask principle.	The Infectable interface grows from 1 method to 3, which increases the contract each implementor must fulfil.
InfectedStatus depends only on the Infectable abstraction, never on any concrete host class (Dependency Inversion Principle). Zero coupling from the status layer to specific types.	Hosts that die instantly on infection (e.g., Undead) must provide no-op implementations for onInfectionTick() and isInfectionActive(), since they never use a status.
Adding a new infectable host only requires implementing the Infectable interface — no additional status class needed (Open-Closed Principle).	
Eliminates class proliferation: one status class replaces three, reducing maintenance burden.	
Duplicate logic (e.g., Parasite-spawning helper) can live in each host's private methods, scoped appropriately rather than duplicated across separate status classes.	

D. How spawners are made configurable across different maps

Design 1: Create separate Hole subclasses for each map (e.g., DeprecatedHole spawning Undead/Slime, OverflowHole spawning Undead/Parasite). Each subclass hardcodes its creature list.

Pros	Cons
Each subclass is self-contained and explicit about what it spawns.	Creates near-identical subclasses that differ only in the creature list, violating DRY.

	Adding a new map with different creatures requires yet another subclass, violating the Open-Closed Principle.
	Expansion logic would need to know which subclass to instantiate for child holes, adding coupling.

Design 2: A single Hole class with a configurable List<Supplier<Actor>> constructor parameter. The creature list is injected at construction time (e.g., via a lambda in the ground creator). Expanded holes inherit the same supplier list from their parent.

Pros	Cons
One Hole class serves all maps with different creature configurations, eliminating subclass proliferation (DRY).	The constructor is more complex (requires a list of suppliers rather than no-arg).
New maps can define their own creature mixtures without creating new classes (Open-Closed Principle).	Cannot use a simple method reference (Hole::new) in the ground creator; requires a lambda.
Expansion naturally inherits the parent's supplier list, so child holes always match their parent's spawning capability.	
Follows the Strategy pattern: creature creation is delegated to Supplier<Actor> strategies injected at construction time.	

Summary

The final design of requirement 4 utilises Design 2 of parts A, B, C, and D stated above to implement spawn effects, infection discovery, infection statuses, and configurable spawners.

Spawn effects are managed centrally by SpawnEffectManager, a singleton registry that maps each creature class to a SpawnEffect functional interface. Whenever any part of the system spawns a creature — whether from a Hole, Vent, tree (REQ3), infection, or Alien Cube sale (REQ2) — it calls SpawnEffectManager.onCreatureSpawned(). The manager looks up and applies the registered effect. This ensures consistent behaviour across all spawning sources and satisfies the DRY and Open-Closed Principles. The three concrete effects (UndeadSpawnEffect, SlimeSpawnEffect, ParasiteSpawnEffect) each implement the SpawnEffect interface with their specific environmental reaction.

The Parasite's infection system is built on the Infectable interface and the engine's asCapability() discovery mechanism. InfectBehaviour scans adjacent tiles using Location.getActorAs(Infectable.class) for actors and Location.getItemsAs(Infectable.class) for items. When a target is found, InfectAction calls target.onInfected(), then removes the Parasite from the map. Each host defines its own infection response: ContractedWorker and

Undead each implement Infectable, as do Lantern and Cookies (which are Items extending GameEntity). The Undead's onInfected() calls unconscious(map) for instant death, while Worker, Lantern, and Cookies each add a generic InfectedStatus that delegates back to their own methods. This polymorphic dispatch eliminates all instanceof checks from the infection system.

Infection effects are handled by a single InfectedStatus class that delegates to the Infectable host. Rather than maintaining separate status classes per host type, the Infectable interface defines three methods: onInfected() for initial setup, onInfectionTick() for per-turn behaviour, and isInfectionActive() for termination. InfectedStatus stores a reference to the Infectable host and calls these methods in its tickStatus() and isStatusActive() implementations. This keeps all infection logic encapsulated within each host class: ContractedWorker deals 1 damage per turn to itself and spawns a Parasite every 5 turns; Lantern drains 1 oil per turn; Cookies reduces its count by 1 per turn and spawns a Parasite each turn. Both Parasite-spawning hosts call SpawnEffectManager after placement, maintaining consistency with the central spawn effect system. This approach eliminates the need for hosts to expose internal state through public getters/setters, achieving better encapsulation than separate status classes.

Spawners are unified through configuration rather than subclassing. Hole takes a List<Supplier<Actor>> in its constructor, injected at map setup time. The 99-deprecated map's holes receive Undead and Slime suppliers; the 20-overflow map's holes receive Undead and Parasite suppliers. When a hole expands, the child hole inherits the parent's supplier list automatically. Vent is a separate Ground subclass for motion-activated spawning of Parasites and Slimes, poisoning all adjacent actors (including the newly spawned creature) on successful spawn. Both spawners use Ability enums (IS_HOLE, ON_FIRE) instead of instanceof for ground-type checks, following OOP best practices.

All new classes follow established patterns: behaviours implement Behaviour<Actor, Action>, the single InfectedStatus implements the engine's Status interface, and entity discovery uses the asCapability() system. This ensures REQ4 integrates cleanly with the existing architecture and provides reusable infrastructure for other REQs.